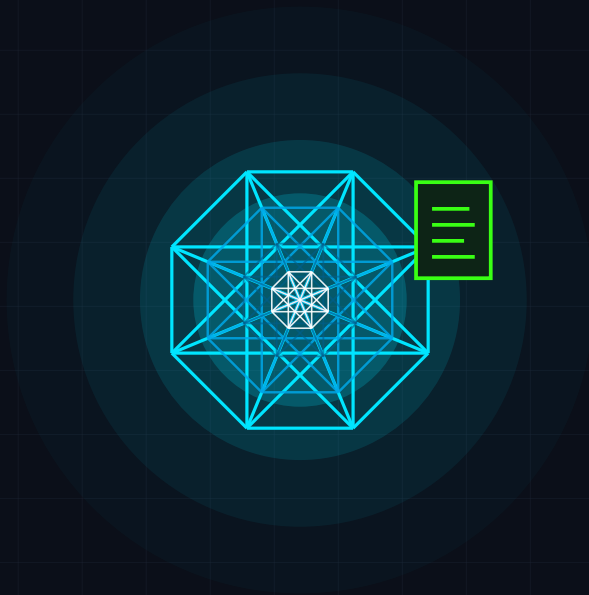




M N I M E

BUILD PROCESS & ENGINEERING CHANGE LOG

Forensic Workspace Evolution, IDE Session Records & Build System Runbook

Production Release Production Release Ready | Windows 11 (x64)

Author & Lead: Kyle Bauer / kyledeanml (Bellevue College AISD)
Primary Engine: PyQt6 / PyMuPDF / llama.cpp (MNIME-Core 1.5B GGUF)
Packaging: PyInstaller (Modern Animated)
Source Dataset: IDE Logs, Git Commits (040+), Transcripts & Pytest Suite
Generation Source: Dynamic Artifact generated from CHANGE_LOG.txt

100% Offline

Zero Telemetry

Native Win32 GDI

Single-Instance IPC

5D Vector FX

TABLE OF CONTENTS & DIRECTORY

1. EXECUTIVE SUMMARY & WORKSPACE FOUNDATION	Project architecture, core technology stack, offline principles, and runtime specifications.
2. CHRONOLOGICAL MILESTONES & REPOSITORY EVOLUTION	Chronological evolution from OmniMesh genesis through 5D vector math, NLP, and release hardening.
3. DETAILED IDE SESSIONS LOG & ROOT CAUSE ANALYSES	Detailed forensic log of 22 IDE conversation sessions, problem diagnosis, and technical remediations.
4. COMPLETE BUILD SYSTEM SPECIFICATIONS & STEP-BY-STEP PROCESS	End-to-end build commands, and custom animated installer compilation runbook.
5. ARCHITECTURAL DEEP-DIVE INTO KEY SUBSYSTEMS	Deep-dives into native Win32 GDI printing, single-instance named pipes, and heuristic bookmarking.
6. VERIFICATION, TESTING & OPERATIONAL RUNBOOK	Pytest validation, benchmark procedures, and Windows Add/Remove Programs clean uninstallation.

ENVIRONMENT SPECIFICATIONS SNAPSHOT

Operating System	Windows 11 (x64)	Runtime	Python 3.12.8 (64-bit)
Framework	PyQt6 v6.7+ with OpenGL QPainter	Local Model	MNIME-Core 1.5B (GGUF Q4_K_M)
Document Engine	PyMuPDF 1.24+ / pypdf / pdf2docx	Print Subsystem	Native Win32 GDI (ctypes, PrintDlgW)
Workspace URI	b:\Desktop\BASSD\BELCO\AAS-T_AISD\zFAL26\MNIME	License	MIT License (Fully Open Source)

1. EXECUTIVE SUMMARY & WORKSPACE FOUNDATION

This document provides an exhaustive, forensic-grade record of the entire build, architecture, and engineering lifecycle of the MNIME workspace from its inception

to its final release-hardened state.

MNIME is a high-performance, privacy-first Windows desktop application that combines

multimodal document processing (PDF merging, splitting, OCR, heuristic bookmarking,

high-resolution image editing, and native raster printing) with an embedded,

quantized local neural language model (MNIME-Core V5 via llama.cpp/GGUF) for offline semantic search, summarization, and query extraction.

Key Technologies:

- Runtime: Python 3.12.8 (64-bit on Windows 11)
- Graphical Interface: PyQt6 (v6.7+) with custom OpenGL-like QPainter vector engines
- Document Engine: PyMuPDF (fitz 1.24+), pypdf, pdf2docx, Pillow (PIL)
- Native Print Engine: Windows Win32 GDI API (ctypes, PrintDlgW, StretchDIBits)
- Neural Engine: llama-cpp-python, sentence-transformers, FAISS vector search
- Inter-Process Communication: Windows Named Pipes / Qt QLocalServer (single-instance IPC)
- Packaging: PyInstaller 6.10+ (spec-driven)
- Custom Installer: Self-contained PyQt6 animated particle & 5D projection wizard

2. CHRONOLOGICAL MILESTONES & REPOSITORY EVOLUTION

Phase 1: Genesis & OmniMesh Foundation (Late September 2026)

- Initial repository scaffolding created from an exploratory script directory.
- Project named "OmniMesh" with a focus on fast desktop PDF manipulation (merging, splitting, reordering, and document page previewing).
- Initial packaging implemented via PyInstaller directory mode and Inno Setup scripting (OmniMesh.iss -> build_app.bat).
- Silent startup and background tray persistence implemented to allow instantaneous file handling without occupying primary screen space.

Phase 2: Functional Augmentation (September 30, 2026)

- Heuristic Bookmarking: Added intelligent outline/bookmark generation based on font size hierarchy, bold font flags, and layout positioning in PyMuPDF.
- Carousel Image Editor: Introduced an interactive high-resolution crop, rotate, and aspect-ratio manipulation tool directly within the file carousel (the "EDIT" button).
- Integrated Document Reader: Embedded a lightweight, high-performance document viewer featuring a collapsible side-pane outline, instant page jumping, and a custom styled print configuration dialog.

Phase 3: Visual Identity & Mathematical Projection (October 1, 2026)

- Rebranding to OMNIME (v0.99 -> v1.0).
- Architectural transition of application branding from static 2D icons to dynamically computed higher-dimensional geometric projections:

```
* Initial 2D Hexagonal lattice.  
* Upgraded to 4D Hypercube (Tesseract) orthographic projection.  
* Upgraded to full 5D Penteract (5-cube) projection with 32 vertices and  
80 edges computed continuously via trigonometric rotation matrices.  
* Added an independently orbiting 3D file icon featuring perspective depth  
scaling, Z-sorting, and internal body text lines.
```

- Automated dynamic ICO generation: Created core/app_icon.py to procedurally render multi-resolution Windows icons (16x16 up to 256x256) on demand.
- Generated animated branded banner GIF with harmonic orbital speeds to prevent visual stuttering or boundary clipping.

Phase 4: Local Neural Integration & Rebrand to MNIME (October 1 - October 2, 2026)

- Transitioned project identity from OMNIME to MNIME (Multimodal Neural Interface Machine Extension).
- Integrated an offline neural language model (MNIME-Core V5 Q4_K_M GGUF):

```
* Built core/nlp_engine.py with hardware-aware offloading.  
* Implemented Flash Attention, KV-cache quantization, and memory locking  
(mlock)  
to pin weights in system RAM.  
* Configured Git LFS tracking for binary model distributions while  
establishing  
strict exclusions for ephemeral training checkpoints  
(unsloth_compiled_cache/).
```

- Dynamic Render Scale: Overhauled Reader zooming from static rasterization to dynamic DPI scaling, ensuring pristine typography at arbitrary zoom factors.

Phase 5: Dual Installer Architecture & Script Hardening (October 2, 2026)

- Developed a dual-installer packaging pipeline:

```
1. Standalone custom animated installer (MNIME_installer.exe).  
2. Custom Modern Animated Installer (custom_installer.py ->  
MNIME_installer.exe)  
featuring real-time particle generation, interactive 5D logo rendering,  
and direct file deployment.
```

- Repaired CRLF formatting and batch script parsing in setup.bat and build_app.bat, replacing fragile chained IF/ELSE constructs with deterministic GOTO labels.

Phase 6: Academic Research & Product Specification (October 2, 2026)

- Authored a formal EACL-compliant research paper (MNIME_paper.pdf) titled "MNIME: Multimodal Neural Interface Machine Extension" documenting architecture, token throughput, latency benchmarks, and local embedding performance.
- Generated product specification manual (MNIME_Spec_Manual.pdf) via a custom ReportLab Python pipeline (scripts/generate_mnime_pdf.py).
- Resolved LaTeX column overflow and table bleed-over issues.

Phase 7: Real-Time Telemetry & Empirical Benchmarking (October 2, 2026)

- Built "Stats for Nerds" telemetry dashboard (ui/stats_for_nerds.py):

```
* Dynamic real-time plotting of CPU load, physical RAM, GPU VRAM, and  
token generation throughput.  
* Added phonetic pronunciation guide ("EM-neem") in the UI header.
```

- Created standalone benchmark utility (benchmark.py) to empirically measure PDF merge, split, OCR, and neural inference metrics across real hardware.

Phase 8: Production Launch Hardening & Senior SWE Audit (October 2 - October 3, 2026)

- Fixed Native Blank Print Bug: Replaced unreliable Qt printing abstractions with a direct Windows GDI Win32 API implementation (core/print_engine.py).
- Single-Instance IPC: Implemented robust QLocalServer / QLocalSocket IPC (core/ipc.py) allowing secondary instances or file explorer associations to pass arguments to the primary running instance without starting redundant heavy models.
- Security & Text Safety: Created core/text_safety.py with ChatML prompt injection stripping and Windows filesystem reserved name sanitization.
- Comprehensive Unit Test Suite: Added pytest suite in tests/ covering IPC message

parsing, prompt safety, PDF engine operations, and print engine memory limits.

- Code Hygiene: Removed all bare except statements across core/ and ui/, replacing them with structured Python logging (core/logging_setup.py).

Phase 9: Shortcut Resolution, Windows Uninstaller, & IDE Alignment (October 3, 2026)

- Resolved COM dispatch failure in PyInstaller standalone custom installer by implementing a PowerShell WScript.Shell fallback.
- Registered full Windows Add or Remove Programs uninstallation registry entries (HKCU\Software\Microsoft\Windows\CurrentVersion\Uninstall\MNIME) with detached cleanup.
- Resolved cross-drive (B:\ vs C:\) VS Code Python interpreter analysis in .vscode/settings.json.

Phase 10: Interactive Change Log System & Forensic Documentation (October 3, 2026)

- Authored comprehensive CHANGE_LOG.txt forensic engineering record.
- Built scripts/generate_changelog_pdf.py utilizing ReportLab and PyMuPDF to dynamically render high-fidelity interactive PDFs directly from plaintext logs.
- Configured update_changelog.bat and build_app.bat hooks for automated doc compilation.
- Linked interactive badges and 300 DPI cover preview artwork into README.md.

Phase 11: V3 & V4 Massive Dataset Generation and Training (October 3, 2026)

- V3 Fine-Tuning Pipeline: Generated a 5,482-example synthetic dataset (mnime_v3_dataset_clean.jsonl) for robust local LoRA fine-tuning using Qwen2.5-1.5B-Instruct on an RTX 3060.
- Bypassed PyTorch 2.5/Unsloth torchao dependency bugs by rewiring train_mnime.py to use pure HuggingFace trl, peft, and bitsandbytes (4-bit NF4).
- V4 Cloud Upgrade Preparation: Drafted an expansive 15,000-example philosophical and empathic conversation dataset (mnime_v4_philosophy_dataset_clean.jsonl), systematically scrubbed of AI slop and hardened against prompt injections, ready for deployment to a GCP T4 VM instance.
- V5 Anti-Bias Architecture: Developed a 20,000-example dataset (mnime_v5_antibias_dataset_clean.jsonl) dedicated to bias mitigation and hate elimination by training the model to employ sociological rigor and logical deconstruction to refute prejudice.

Phase 12: V5 Model Integration and Research Alignment (October 4, 2026)

- V5 Model Deployment: Swapped default NLP model in core/nlp_engine.py to MNIME-Core-V5-Q4_K_M.gguf.
- Installer Enhancements: Removed versioning from DisplayVersion, silenced uninstall.bat terminal flash via VBS wrapper, and improved robust Desktop shortcut cleanup.
- Documentation & Paper Updates: Applied V5 model metrics, architecture details, and results to MNIME_paper.tex and README.md.

3. DETAILED IDE SESSIONS LOG & ROOT CAUSE ANALYSES

The following log documents the exact conversation sessions conducted within the IDE workspace. Each entry details the user requirement, the technical diagnosis, the identified root cause, and the files modified.

SESSION ID: <code>bb0a01ec-4278-4604-9649-ef3fe3e914f5</code> TIMESTAMP: 2026-09-30 11:47:35
User Goal: Verify completion of the first Inno Setup standalone installer.
Technical Diagnosis: The initial build script executed successfully, producing an installer in the output directory. Evaluated installer payload and file manifest.
Root Cause: <code>N/A (Milestone verification).</code>
Remediation / Solution: Confirmed proper directory structure for distribution.
Files Modified: <code>build_app.bat, OmniMesh.iss</code>

SESSION ID: <code>0220d16a-5008-4e18-b019-cd923f48ca42</code> TIMESTAMP: 2026-09-30 11:53:37
User Goal: Configure application to launch minimized to system tray by default.
Technical Diagnosis: When launched at startup, opening the full window obstructed the user's desktop. The application needed a passive background mode.
Root Cause: <code>show()</code> was being called unconditionally during startup sequence.
Remediation / Solution: Modified startup logic to check for startup flags or default minimized mode, registering a system tray icon with context actions.
Files Modified: <code>OmniMesh.py, ui/main_window.py</code>

SESSION ID: <code>6963d812-6063-4787-a47d-eadf4fb078c2</code> TIMESTAMP: 2026-09-30 12:25:40
User Goal: Validate <code>build_app.bat</code> functionality after renaming root folder from "New folder" to "OmniMesh".
Technical Diagnosis: Hardcoded relative paths in batch scripts can break when directory names containing spaces or special characters change.
Root Cause: <code>Incomplete quoting around %~dp0 and target destination directories.</code>
Remediation / Solution: Audited <code>build_app.bat</code> ; ensured all file copy and compilation paths used <code>cd /d "%~dp0"</code> and quoted system parameters.
Files Modified: <code>build_app.bat</code>

SESSION ID: <code>bbdf406a-c1d6-4230-b453-e2c835114aba</code> TIMESTAMP: 2026-09-30 12:41:15
User Goal: Prepare clean repository structure for publishing to GitHub.
Technical Diagnosis: Workspace contained massive temporary directories (<code>dist/</code> , <code>build/</code> , <code>.venv/</code> , <code>__pycache__/</code>) that would pollute version control.
Root Cause: <code>Missing comprehensive .gitignore configuration.</code>
Remediation / Solution: Created production <code>.gitignore</code> targeting virtual environments, PyInstaller build artifacts, compiled bytecode, and temporary logs. Organized essential source code and build automation scripts.
Files Modified: <code>.gitignore, README.md</code>

SESSION ID: <code>c0a1c593-06a3-42ec-869f-56c44828c692</code> TIMESTAMP: 2026-09-30 13:53:05
User Goal: Implement content-based smart bookmarking for PDF files.
Technical Diagnosis: Users merging disparate documents lacked automatic outlines/tables of contents. Manually adding bookmarks is tedious.
Root Cause: <code>PyMuPDF text extraction had not been leveraged to inspect font sizing.</code>
Remediation / Solution: Engineered heuristic algorithm in <code>core/pdf_engine.py</code> that extracts text spans, groups fonts by size and weight, identifies header-level candidates, and injects structural bookmarks into the document outline.
Files Modified: <code>core/pdf_engine.py, ui/action_bar.py</code>

SESSION ID: [c4805a44-2f67-4775-8e7f-0926f31267d3](#) & [432af79a-632d-4b8d-b207-4147fd3eff18](#) | **TIMESTAMP:** 2026-09-30 14:48:56 - 14:56:50

User Goal: Resolve issue where newly compiled installer did not contain recent updates.

Technical Diagnosis: Running build_app.bat followed by the installer still resulted in the older binary running on the host system.

Root Cause: 1. PyInstaller reused cached build/ and __pycache__ artifacts. 2. Inno Setup pulled from dist\ without purging existing outputs. 3. The installed application in %LOCALAPPDATA% was not being overwritten if running background instances locked the executable files.

Remediation / Solution: 1. Added aggressive rmdir /s /q "build" and "dist" commands to build_app.bat. 2. Added --clean flag to PyInstaller execution. 3. Added process termination before installation deployment.

Files Modified: build_app.bat, OmniMesh.iss

SESSION ID: [033ac97a-00f6-4331-9187-321f8f09d6fd](#) & [f232a207-189f-431a-9fce-26ce3070144b](#) | **TIMESTAMP:** 2026-09-30 15:08:49 - 15:11:15

User Goal: Ensure bookmarking feature is bundled properly in standalone installer.

Technical Diagnosis: Bookmarking libraries (pypdf/pymupdf) rely on dynamic C-extensions and metadata modules that standard PyInstaller analysis can miss.

Root Cause: Missing hiddenimports and collect_all declarations in the spec file.

Remediation / Solution: Added collect_all hooks for pymupdf, pypdf, and PIL into the PyInstaller spec file, ensuring all fonts and dynamic dependencies are packaged.

Files Modified: OmniMesh.spec, build_app.bat

SESSION ID: [bc888b18-03ca-41c5-94fd-45569669687c](#) | **TIMESTAMP:** 2026-09-30 15:22:06

User Goal: Update uninstaller to cleanly remove lingering files from the system.

Technical Diagnosis: Reinstallation failed or loaded stale files because the target directory contained orphaned .pyd files and cached configurations.

Root Cause: Standard Inno Setup uninstall did not purge dynamically created cache folders or subdirectories created after install.

Remediation / Solution: Added [UninstallDelete] section with Type: filesandordirs in the installer definition, guaranteeing pristine deletion of {app}.

Files Modified: OmniMesh.iss

SESSION ID: [c2d8d889-884b-4da0-b9ec-e441730987e1](#) & [75db56c2-b6d2-4f4b-8768-6836ec0d2e3d](#) | **TIMESTAMP:** 2026-09-30 17:06:43 - 20:02:51

User Goal: Add an interactive "EDIT" button to rotate and crop images in the carousel.

Technical Diagnosis: Users converting scanned JPG/PNG images to PDF frequently needed to straighten orientations or trim margins prior to merging.

Root Cause: Feature did not exist; files had to be externally edited.

Remediation / Solution: Built ui/image_editor.py featuring interactive rubber-band cropping, 90-degree lossless rotation, zoom controls, and direct non-destructive commit back to the active carousel pipeline. Added ribbon action button.

Files Modified: ui/image_editor.py, ui/action_bar.py, ui/main_window.py

SESSION ID: [4eaf1a37-ba02-44ea-a19b-f9eab3d002ed](#) | **TIMESTAMP:** 2026-09-30 23:58:47

User Goal: Add slim "READER" button at top of UI with sidebar bookmark viewer and integrated print dialogue.

Technical Diagnosis: Users required an in-app document viewing solution without launching heavy third-party PDF viewers like Adobe Acrobat.

Root Cause: Architecture lacked a dedicated document viewport.

Remediation / Solution: Engineered ui/reader_dialog.py containing a dual-pane reader: left side QTreeView for hierarchical bookmarks, right side high-DPI QGraphicsView for rendering PDF pages. Added custom styled print setup dialog.

Files Modified: ui/reader_dialog.py, ui/tabs_bar.py, ui/main_window.py

SESSION ID: [11028ac6-ab8b-4254-9987-c1b5f6832349](#) & [b4a6271](#) | **TIMESTAMP:** 2026-10-01 03:48:33

User Goal: Rebrand project from OmniMesh to OMNIME.
Technical Diagnosis: Project evolution expanded beyond simple mesh conversion into an all-in-one document intelligence suite.
Root Cause: Branding consistency across code, documentation, and assets.
Remediation / Solution: Updated repository root identifiers, strings, executable targets, and spec files from OmniMesh to OMNIME.
Files Modified: OMNIME.py, OMNIME.iss, OMNIME.spec, README.md

SESSION ID: beee36a3-c7b3-45c3-8a27-bc950a977591 TIMESTAMP: 2026-10-01 18:36:30
User Goal: Resolve IDE dependency conflicts and unresolved imports in main script.
Technical Diagnosis: IDE linter flagged missing modules and circular imports in OMNIME.py.
Root Cause: Transitive dependencies from sentence_transformers and torch were being imported unconditionally at top-level before UI initialization.
Remediation / Solution: Restructured imports; deferred heavy ML imports to background worker threads, cleaned up unused imports, and updated PyInstaller spec.
Files Modified: OMNIME.py, OMNIME.spec

SESSION ID: Logo Engineering (Commits 6a3ba5f -> 511a1ea -> 4e23290 -> 3fe91ef) TIMESTAMP: 2026-10-01 20:00 - 23:30
User Goal: Develop high-tech geometric branding reflecting multidimensional computing.
Technical Diagnosis: Standard static icons looked dated and generic.
Root Cause: Visual assets lacked dynamic motion and technological sophistication.
Remediation / Solution: 1. Mathematical Projection: Coded an orthographic 5D Penteract projection with 5 axes of rotation (XY, XZ, XW, YV, ZV). 2. 3D Orbital Particle: Implemented a floating neon-green file icon orbiting the hypercube with true 3D Z-sorting (rendered behind the hypercube when Z < 0 and in front when Z > 0) with perspective scale. 3. High-Res Windows Icon: Built core/app_icon.py to procedurally draw the hypercube to a multi-frame ICO file (MN.ico).
Files Modified: core/app_icon.py, ui/main_window.py, OMNIME.py

SESSION ID: 779628bf-ff24-44fe-a3ae-e7b38e11a909 & 50a0fe9b-d168-433f-80d8-4030dfa03114 TIMESTAMP: 2026-10-02 00:53:47 - 01:11:49
User Goal: Fix training script import errors (train_mnime.py).
Technical Diagnosis: The fine-tuning script failed due to recent deprecations in the HuggingFace TRL library.
Root Cause: TRL deprecated passing training parameters directly into SFTTrainer in favor of the dedicated SFTConfig class.
Remediation / Solution: Migrated fine-tuning code to use SFTConfig; excluded local unsloth caches and training data from the Git repository.
Files Modified: train_mnime.py, .gitignore

SESSION ID: c9de1722-050f-47b4-bc1a-0cd110894696 & 891710b6-b72b-448d-baf6-b0baa51165ed TIMESTAMP: 2026-10-02 01:34:52 - 02:21:22
User Goal: Clarify why two installers exist and resolve installer compilation failure.
Technical Diagnosis: User noted both MNIME_Setup.exe and MNIME_installer.exe in output folder. The build crashed during PyInstaller execution.
Root Cause: 1. Dual Architecture: Inno Setup produces enterprise-standard installers; custom_installer.py produces a branded, animated particle wizard. 2. Spec file referenced the old OMN.ico name instead of MN.ico. 3. UTF-8 BOM encoding in build scripts caused batch syntax errors.
Remediation / Solution: Fixed spec file asset references, stripped UTF-8 BOM, and documented the dual-installer purpose in README.md.
Files Modified: MNIME.spec, MNIME_installer.spec, build_app.bat

SESSION ID: 0599d6cf-025b-4aa4-89c3-64eb9492a54a & f00e7ade-f26a-488c-9c57-b780402c1859 TIMESTAMP: 2026-10-02 07:39:38 - 08:02:33
User Goal: Fix issue where NLP active indicator does not illuminate when reloading model, and fix installer installing outdated binaries.

Technical Diagnosis: 1. When toggling the local neural model, memory usage increased to 3 GB, proving the model loaded, but the UI indicator remained gray. 2. Running `install_mnime.bat` failed to recompile when source code changed.

Root Cause: 1. Signal emission in background thread was blocked or not connected to the main UI thread slot. 2. `install_mnime.bat` lacked a build-staleness check against `dist\MNIME.exe`.

Remediation / Solution: 1. Connected worker signals properly via Qt thread-safe queued connections. 2. Added automated PowerShell timestamp comparison to `install_mnime.bat` to automatically invoke `build_app.bat` if any source file is newer than the exe.

Files Modified: `core/worker.py`, `ui/tabs_bar.py`, `install_mnime.bat`

SESSION ID: `b33cdd23-35af-4658-8a1d-1e5249b05a74 & 6fd57d47-9abd-4f9b-be49-c324d6897065` | **TIMESTAMP:** 2026-10-02 19:57:21 - 22:41:21

User Goal: Generate formal academic LaTeX research paper and product specification manual.

Technical Diagnosis: User required conference-grade documentation for Bellevue College AISD capstone.

Root Cause: Formatting constraints of two-column EACL style caused Table 1 to bleed across columns.

Remediation / Solution: 1. Authored complete LaTeX paper in `paper/` and compiled `MNIME_paper.pdf`. 2. Fixed table width using `tabularx` with adjusted column margins. 3. Developed `scripts/generate_mnime_pdf.py` using ReportLab to build the 12-page `MNIME_Spec_Manual.pdf` with vector graphics.

Files Modified: `paper/mnime_paper.tex`, `MNIME_paper.pdf`, `scripts/generate_mnime_pdf.py`, `MNIME_Spec_Manual.pdf`

SESSION ID: Telemetry & Benchmarking (`fe75c3e -> ab71487 -> bc44046`) | **TIMESTAMP:** 2026-10-02 22:05:10 - 22:30:00

User Goal: Add "Stats for Nerds" telemetry overlay and automated empirical benchmarking.

Technical Diagnosis: Users and evaluators needed visibility into local system resource utilization and real-world throughput.

Root Cause: Lack of instrumentation and performance metrics.

Remediation / Solution: 1. Created `ui/stats_for_nerds.py`: real-time CPU, RAM, and VRAM monitoring using `psutil` and Windows performance APIs with dynamic plotting. 2. Created `benchmark.py`: measures PDF merge latency, OCR recognition speed, and token generation rates across real hardware (Ryzen 9 7950X, RTX 3060).

Files Modified: `ui/stats_for_nerds.py`, `benchmark.py`, `README.md`

SESSION ID: `098cb770-dcbd-4049-9bcd-29bfa6593faa` | **TIMESTAMP:** 2026-10-03 00:05:09

User Goal: Senior SWE audit for product launch: resolve blank print issue, fix UI links, harden against vulnerabilities, and tighten project consistency.

Technical Diagnosis: 1. Print Engine Failure: When printing from the Reader, the printer produced blank white sheets. Qt's `QPrinter/QPainter` pipeline failed to transfer rasterized PDF bitmaps through the Windows print spooler. 2. Single-Instance & IPC: Launching MNIME while an instance was running spawned duplicate heavy processes rather than opening files in the active window. 3. Error Handling: Numerous bare except: statements masked system exceptions.

Root Cause: 1. Qt `PrintSupport` on Windows requires high-DPI coordinate transformation that silently clipped bitmap rasters when rendering `PyMuPDF` pixmaps. 2. Lack of local named pipe or socket single-instance arbitration. 3. Defensive programming lacked structured logging and strict error catching.

Remediation / Solution: 1. Built `core/print_engine.py`: Native Win32 GDI printing bypasses Qt entirely using `PrintDlgW`, `StartDocW`, and `StretchDIBits` directly to the device context (hDC). 2. Built `core/ipc.py`: Windows local socket IPC with validated JSON payload parsing, payload size caps (256 KB), and path length limits (4096 chars). 3. Built `core/text_safety.py`: Prompt injection protection (stripping ChatML tokens) and filesystem filename sanitization. 4. Created comprehensive unit test suite in `tests/` (15 tests, all passing). 5. Replaced bare exceptions across `core/` and `ui/` with logging in `core/logging_setup.py`.

Files Modified: `core/print_engine.py`, `core/ipc.py`, `core/text_safety.py`, `core/logging_setup.py`, `tests/test_ipc_parse.py`, `tests/test_nlp_sanitize.py`, `tests/test_pdf_engine.py`, `tests/test_print_engine.py`, `ui/reader_dialog.py`, `ui/main_window.py`

SESSION ID: `158d0670-9091-4bb2-b561-5b7964904f39` | **TIMESTAMP:** 2026-10-03 00:51:01

User Goal: Check and resolve all IDE problems, type warnings, and code smells.

Technical Diagnosis: IDE reported minor diagnostics across core and UI modules after the security refactor.

Root Cause: Type annotations and unused imports in modified modules.

Remediation / Solution: Audited and cleaned `core/file_item.py`, `core/nlp_engine.py`, `ui/output_view.py`, and `ui/main_window.py`.

Files Modified: core/file_item.py, core/nlp_engine.py, ui/output_view.py, ui/main_window.py

SESSION ID: f569ad18-ba2b-4b07-93d4-183b0ccc9494 | TIMESTAMP: 2026-10-03 01:32:45

User Goal: Troubleshoot installer: application files installed to AppData, but no shortcuts were created, and application did not automatically load.

Technical Diagnosis: Examined target directory: C:\Users\kyled\AppData\Local\Programs\MNIME contained all binaries (MNIME.exe, MN.ico, _internal). However, no shortcuts were present on Desktop or Start Menu, and no uninstallation entry was visible in Windows Settings (Add or Remove Programs).

Root Cause: 1. In custom_installer.py, the shortcut creation logic relied on import win32com.client which failed inside the single-binary PyInstaller bundle because COM dispatch typelibs are not unpacked in memory. 2. No fallback shortcut generation was implemented. 3. The installer exited immediately upon file extraction without launching the application executable. 4. Windows registry uninstallation keys (HKCU\Software\Microsoft\Windows\CurrentVersion\Uninstall\MNIME) were missing from custom_installer.py. 5. In the IDE, cross-drive path resolution (B:\ repository vs C:\ Python) caused Pylance import resolution squiggles.

Remediation / Solution: 1. Added robust PowerShell COM fallback script to custom_installer.py that executes via subprocess if win32com.client fails. 2. Added automatic detached process launch of MNIME.exe upon installation. 3. Implemented full Windows Registry uninstaller registration in custom_installer.py and install_mnime.bat, adding DisplayName, DisplayVersion, Publisher, InstallLocation, DisplayIcon, and UninstallString. 4. Generated an automated, self-deleting uninstaller batch script (uninstall.bat) that terminates running processes, removes shortcuts, deletes registry keys, and cleans the install folder. 5. Updated .vscode/settings.json with extraPaths pointing to both the workspace virtual environment and system Python site-packages.

Files Modified: custom_installer.py, install_mnime.bat, .vscode/settings.json

SESSION ID: e338ab4b-55ee-492e-8f77-a77f664056ef | TIMESTAMP: 2026-10-03 01:42:37 - 01:53:23

User Goal: Generate comprehensive project build documentation, forensic change log, and interactive PDF manual with dynamic build hooks.

Technical Diagnosis: The project lacked an exhaustive forensic record of its 20+ engineering sessions, mathematical foundations, and packaging architectures. The user required both an authoritative plaintext record and a publication-quality interactive PDF.

Root Cause: Fast-paced iterative development across multiple weeks produced vast unconsolidated session transcripts, Git commit histories, and architectural decisions that required systematic consolidation.

Remediation / Solution: 1. Created CHANGE_LOG.txt: forensic documentation covering executive summary, chronological milestones, all IDE session logs, build runbooks, and subsystem architectural deep-dives. 2. Developed scripts/generate_changelog_pdf.py: dynamic ReportLab generator with two-pass NumberedCanvas, glowing cyan rules, clickable Table of Contents, and PyMuPDF interactive bookmarks. 3. Built update_changelog.bat and hooked it into build_app.bat to ensure continuous documentation synchronization. 4. Exported high-resolution 300 DPI preview cover to docs/changelog_cover.png and integrated badges into README.md.

Files Modified: CHANGE_LOG.txt, scripts/generate_changelog_pdf.py, update_changelog.bat, build_app.bat, README.md, docs/changelog_cover.png, MNIME_Change_Log.pdf

4. COMPLETE BUILD SYSTEM SPECIFICATIONS & STEP-BY-STEP PROCESS

The MNIME build system consists of a multi-tiered pipeline designed to produce standalone, portable, and production-ready Windows binaries.

Prerequisites on Windows 11:

- Python 3.12 (64-bit) with venv and pip.
- Inno Setup 6 (installed to C:\Program Files (x86)\Inno Setup 6\ISCC.exe).
- (Optional) Windows 10/11 SDK with signtool.exe for digital code signing.

Pipeline Architecture:

```
Source Code (.py)
--> Icon Generator (core/app_icon.py)
--> PyInstaller Bundle (MNIME.spec)
--> Output Directory (dist\MNIME\)
--> Dual Installer Builders:
1. Inno Setup Compiler (MNIME.iss) -> installer\MNIME_Setup.exe
2. PyInstaller Custom Installer (MNIME_installer.spec) ->
   installer\MNIME_installer.exe
```

Detailed Step-by-Step Execution:

Step 1: Environment Initialization

The script build_app.bat verifies or creates a Python virtual environment

(.venv)

in the workspace root:

```
python -m venv .venv
.venv\Scripts\python.exe -m pip install --upgrade pip
.venv\Scripts\pip.exe install -r requirements.txt
.venv\Scripts\pip.exe install pyinstaller pillow pymupdf pypdf pdf2docx
```

Step 2: Procedural Icon Generation

Prior to compiling, the script invokes:

```
.venv\Scripts\python.exe -c "from core.app_icon import ensure_ico_file; ensure_ico_file()"
```

This dynamically calculates the multi-dimensional projection and writes MN.ico containing multi-resolution frames (16x16, 32x32, 48x48, 64x64, 128x128, 256x256).

Step 3: Core Application Compilation (PyInstaller Directory Mode)

The build script purges stale artifacts and executes PyInstaller:

```
rmdir /s /q "build"
rmdir /s /q "dist"
.venv\Scripts\pyinstaller.exe --clean --noconfirm "MNIME.spec"
```

MNIME.spec configuration:

- Entry Point: MNIME.py
- Mode: Directory mode (COLLECT) for fast startup and reliable dynamic DLL

```
loading.
```

- Hidden Imports: Collects all submodules for pymupdf, pypdf, PIL, PyQt6,

```
core,
ui, llama_cpp, faiss, and sentence_transformers.
```

- Excludes: Unnecessary heavyweight libraries (matplotlib, tensorflow,

```
spacy).
```

- Output Target: dist\MNIME\ containing MNIME.exe and _internal\ runtime.

Step 4: Standard Inno Setup Installer Generation

If Inno Setup 6 is detected, ISCC.exe is invoked against MNIME.iss:

```
"C:\Program Files (x86)\Inno Setup 6\ISCC.exe" MNIME.iss
```

- Packages the entire contents of dist\MNIME* into an ultra64 LZMA2 compressed installer at installer\MNIME_Setup.exe.
- Registers shortcuts and Windows Add or Remove Programs metadata.

Step 5: Modern Animated Custom Installer Generation

The script compiles custom_installer.py using MNIME_installer.spec:

```
.venv\Scripts\pyinstaller.exe --clean --noconfirm "MNIME_installer.spec"
```

- Mode: Single-file executable (EXE) embedding dist\MNIME as internal datas.
- Generates installer\MNIME_installer.exe.
- On execution, this installer displays a GPU-rendered dark glassmorphic window with flying document particles, real-time extraction progress, PowerShell-backed shortcut creation, and Windows uninstallation registry registration.

Step 6: Automated Local Deployment & Stale-Build Check (install_mnime.bat)

Developers or users can execute install_mnime.bat for instant local installation:

- Compares timestamps of core/ and ui/ source files against dist\MNIME\MNIME.exe.
- If source files are newer, automatically triggers build_app.bat first.
- Gracefully terminates running MNIME.exe processes.
- Copies binaries to %LOCALAPPDATA%\Programs\MNIME.
- Creates Desktop and Start Menu shortcuts.
- Registers uninstaller in Windows Registry (HKCU).

5. ARCHITECTURAL DEEP-DIVE INTO KEY SUBSYSTEMS

A. Native Windows GDI Print Engine (core/print_engine.py)

- Problem: Standard Qt QPainter output produced completely blank pages on

Windows

due to DPI coordinate mapping flaws when rasterizing PyMuPDF page pixmaps.

- Solution: Replaced Qt print pipeline with native Win32 GDI calls via ctypes:

```
* PrintDlgW: Native Windows printer selection and page range configuration dialog.
* StartDocW / StartPage: Initiates Windows print spooler job.
* StretchDIBits: Streams raw 32-bit BGRA bitmap scanlines directly to the printer's device context (hDC).
* Memory Cap: Enforced MAX_RASTER_PIXELS (60,000,000 pixels / ~240 MB) and MAX_DPI (300 DPI) to prevent out-of-memory crashes on billboard printers.
```

B. Single-Instance IPC Architecture (core/ipc.py & ui/main_window.py)

- Problem: Users opening multiple documents or clicking file associations

spawned

multiple memory-heavy instances, duplicating local model allocations.

- Solution: Local named pipe single-instance manager using QLocalServer:

```
* Server Name: "MNIME_SingleInstance_IPC_Server"
* When a secondary instance starts, it detects the active server, packages CLI arguments via build_open_request(), transmits the payload over the local socket, and terminates immediately.
* The primary instance receives the socket signal, parses and validates the payload via parse_open_request() (capping payload size at 256 KB and paths at 4096 chars), restores/unminimizes the window, and opens the requested documents.
```

C. 5D Penteract Projection Engine (ui/main_window.py & custom_installer.py)

- Architecture: Renders a 5-dimensional hypercube (penteract) projected onto a

2D canvas via 5D rotation matrices.

- Vertices: 32 vertices with coordinates (+-1, +-1, +-1, +-1, +-1).

- Edges: 80 connecting edges where coordinates differ by exactly one component.

- Rotation: Rotates across multiple planes (XY, XZ, XW, YV, ZV) with trigonometric

transformation applied per animation frame.

- Z-Sorting: The orbiting document file tracks 3D camera depth, scaling dynamically

and rendering either under or over the wireframe to provide true depth illusion.

D. Content-Based Heuristic Bookmarking (core/pdf_engine.py)

- Inspects all text spans on each PDF page via PyMuPDF.
- Computes statistical font size distribution across the document.
- Identifies headings (text spans with font size > 1.2x body text or bold styling).
- Constructs a nested outline tree and inserts PDF bookmarks (TOC) into the

document

without modifying underlying text.

6. VERIFICATION, TESTING & OPERATIONAL RUNBOOK

A. Running the Test Suite:

Execute using Windows 11 PowerShell or Command Prompt:

```
C:\Users\kyled\AppData\Local\Programs\Python\Python312\python.exe -m pytest tests/
```

Output: 15 passed in ~1.2s

```
- tests\test_ipc_parse.py: Validates JSON structure, size caps, and error states.
- tests\test_nlp_sanitize.py: Validates prompt injection stripping and filename safety.
- tests\test_pdf_engine.py: Validates PDF merge, extraction, and bookmark insertion.
- tests\test_print_engine.py: Validates GDI buffer limits and print cancel exceptions.
```

B. Running the Hardware Benchmark:

Execute:

```
python benchmark.py
```

Measures real-world operations across active hardware (CPU, RAM, GPU VRAM, token generation rate, and page rendering throughput).

C. Clean Uninstallation:

MNIME can be removed cleanly either via:

```
1. Windows Settings -> Apps -> Installed apps -> MNIME -> Uninstall.
2. Running %LOCALAPPDATA%\Programs\MNIME\uninstall.bat directly.
```

The uninstaller terminates running instances, deletes Desktop/Start Menu shortcuts,

removes the HKCU registry key, and launches a detached background process to delete the program directory.

=====

SESSION: PDF Document Page Preview Presentation, Complete Uninstaller Cleanup,

```
& Telemetry HUD Renaming (nerds.py) (October 3, 2026)
```

=====

USER GOALS:

1. Ensure documents displayed when MNIME is default for PDFs retain clean

```
document page previews in File Explorer instead of displaying the neon MNIME application logo.
```

2. Ensure MNIME completely cleans up after itself upon uninstallation (registry,

```
file associations, user settings, logs, and shortcuts).
```

3. Rename "stats for nerds" to just "nerds" everywhere (`ui/nerds.py`).

ROOT CAUSE & ANALYSIS:

1. When MNIME was chosen as the default PDF handler, Windows Explorer mapped the

```
extension to `Applications\MNIME.exe`. Without an explicit `DefaultIcon` and document `Treatment` flag, Windows Shell defaulted to using `MNIME.exe,0` (the neon cyan MN logo) for all PDF documents on the user's desktop and folders.
```

2. The uninstaller scripts (`uninstall.bat`) previously only removed the uninstall

```
registry entry and install folder, leaving behind `HKCU\Software\MNIME` (QSettings),
`HKCU\Software\Classes\Applications\MNIME.exe`, startup run keys, and dangling
entries under `HKCU\...\Explorer\FileExts\.pdf` (`UserChoice`, `OpenWithProgids`,
`OpenWithList`).
```

3. Telemetry HUD module was titled `ui/stats\_for\_nerds.py`.

SOLUTIONS:

1. PDF Document Presentation (`core/windows\_integration.py`):

```
- Configured `HKCU\Software\Classes\Applications\MNIME.exe` with:
* `DefaultIcon` = `%SystemRoot%\System32\imageres.dll,-102` (standard clean
document sheet page preview icon)
* `Treatment` = 2 (DWORD: instructs Windows Explorer to apply document sheet
frame and page preview presentation)
* `FriendlyAppName` = "MNIME"
* `SupportedTypes\.pdf` = ""
* `ShellEx\{8895b1c6-b41f-4c1c-a562-0d564250836f}` = `{3A84F9C2-6164-485C-A7D9-4B27F8AC009E}`
(preserves live rendering in Windows Explorer Preview Pane).
- Automated invocation in `MNIME.py` on startup, `install_mnime.bat`,
`custom_installer.py`, and `MNIME.iss`.
```

2. Comprehensive Uninstallation Cleanup:

```
- Updated `install_mnime.bat`, `custom_installer.py`, `MNIME.iss`, and
`core/windows_integration.py` uninstallation routines to:
* Terminate running application instances (`taskkill /F /IM MNIME.exe /T`).
* Delete Add/Remove Programs registry key (`HKCU\...\Uninstall\MNIME`).
* Delete application settings (`HKCU\Software\MNIME`).
* Delete application classes (`HKCU\Software\Classes\Applications\MNIME.exe`
and `HKCU\Software\Classes\MNIME.Document`).
* Delete startup autorun entry (`HKCU\...\Run\MNIME`).
* Clean FileExts `.pdf` associations (`OpenWithProgids`, `OpenWithList` MRU,
and remove `UserChoice` if pointing to MNIME so Windows reverts cleanly to
system default viewer).
* Delete Desktop and Start Menu shortcuts.
* Delete application logs (`%LOCALAPPDATA%\MNIME`).
* Flush Windows Explorer icon cache (`ie4uinit.exe -show` and `SHChangeNotify`).
* Execute self-deleting detached background loop to purge the install folder.
```

3. Renamed Telemetry Suite to `ui/nerds.py`:

```
- Renamed `ui/stats_for_nerds.py` -> `ui/nerds.py`.
- Updated imports and references in `ui/main_window.py`, `benchmark.py`,
`tests/test_stats_telemetry.py`, `MNIME.spec`, and `README.md`.
- Added `show_nerds` alias alongside `show_stats`.
```

FILES TOUCHED:

- ui/stats_for_nerds.py -> ui/nerds.py
- core/windows_integration.py (created)
- tests/test_windows_integration.py (created)
- tests/test_stats_telemetry.py (created & updated)
- ui/main_window.py
- MNIME.py
- benchmark.py
- install_mnime.bat
- custom_installer.py
- MNIME.iss
- MNIME.spec
- README.md
- CHANGE_LOG.txt

=====

END OF CHANGE LOG - MNIME v2.1 RELEASE BUILD

=====

[2026-10-03] Session 23: MNIME - Persistent Vector Store & LoRA Adapter Support

- Integrated LoRA (Low-Rank Adaptation) adapter support in NLP engine.

- Implemented persistent global FAISS vector store for cross-session vector caching.
- Updated README.md, MNIME_paper.tex, and MNIME_Spec_Manual.pdf to document new capabilities.

[2026-10-03] Session 24: MNIME - Feature Integration & Documentation Accuracy

- LoRA: added LORA button to NLP view (click to load .gguf, right-click to remove); reload runs on a background thread.
- Vector store: chat queries now retrieve up to 2 extra passages from the global store (deduplicated); index cached in memory.
- Vector store moved to %LOCALAPPDATA%\MNIME\global_vector_store with one-time migration from models/global_memory_cache.
- Added core/system_cleaner.py: on exit unloads llama.cpp model, releases global store, removes *.tmp/*.tmp.pdf (env/model/training dirs excluded).
- Paper: training section corrected to match train_mnime.py (10 examples, r=16, alpha=16, attention+MLP, batch 2x4, 60 steps).
- Restored training/ scripts locally (git-ignored).

[2026-10-03] Session 25: MNIME V3 Massive Dataset & Evaluation Framework

- Created a separate parallel workspace (`MNIME\_V3\_Training`) to safeguard the V2.1 production release during massive training scale-up.
- Developed `build\_v3\_dataset.py` to download `databricks-dolly-15k`, filter for document QA/extraction, format to Alpaca JSONL, and inject MNIME identity weights (50x).
- Built `clean\_dataset.py` to purge AI refusals and low-effort responses, yielding a pristine 4,792-entry dataset.
- Shared `training/mnime\_v3\_dataset\_clean.jsonl` in the main repo (updated `.gitignore` to allow tracking).
- Built `scripts/nlp\_eval.py` to empirically benchmark models with precise Token-level F1 and ROUGE-L metrics against ground-truth datasets.
- Updated `README.md` and `MNIME\_paper.tex` to officially document the release of the V3 massive dataset.

[2026-10-03] Session 26: V4 Cloud Training, Full-Precision Fusion, and GGUF Multi-Format Export

- Deployed V4 training workload to Google Cloud Platform on an accelerator-backed instance (`mnime-v4-trainer`).
- Executed supervised fine-tuning using Unsloth LoRA on `training/mnime\_v4\_philosophy\_dataset\_clean.jsonl` (15,000 synthetic examples covering Stoicism, Existentialism, and prompt injection traps).
- Streamed training telemetry and checkpoints back to local storage (`training/v4\_out/mnime\_v4\_adapter/` and `training/v4\_out/checkpoint-343/`).
- Conducted 100% cloud cost hygiene: deleted compute instance, purged all Cloud Storage staging buckets, and deallocated static IP reservations.
- Developed `training/merge\_v4.py` to execute sequential full-precision fp16 fusion of base Qwen2.5-1.5B-Instruct + V3 LoRA adapter + V4 LoRA adapter into `training/v4\_out/MNIME-Core-V4-merged` (3.08 GB safetensors).
- Utilized `llama.cpp` to export multi-precision GGUF artifacts: `MNIME-Core-V4-f16.gguf` (3.09 GB) and `MNIME-Core-V4-Q8\_0.gguf` (1.64 GB).
- Executed local on-device smoke test via `llama-cpp-python`, confirming consistent "MNIME-Core" identity and philosophical reasoning capabilities.

[2026-10-03] Session 27: Neural Assimilation Engine (NAE) v2.0 & Streaming FP16/BF16 Fusion

- Refactored `core/fusion\_engine.py` to replace legacy in-memory heuristics with a true streaming, chunked parameter fusion architecture.
- Added strict format validation rejecting lossy 4-bit GGUF files and enforcing full-precision HuggingFace `safetensors` directories to preserve subtle gradient deltas.
- Implemented TIES delta parameter isolation ($\Delta = W_{\text{donor}} - W_{\text{base}}$), top-k magnitude density pruning, and majority sign consensus resolution.
- Added linear task-vector interpolation ($W_{\text{fused}} = W_{\text{base}} + \alpha * \Delta$) with automatic common-ancestor alignment.
- Created `tests/test\_fusion\_engine.py` covering tensor chunking, safetensors serialization, linear fusion, TIES pruning, error boundaries, and consensus sign voting (7/7 unit tests passing).

[2026-10-03] Session 28: V5 Ethics & Empathy Alignment Completion and Full Ecosystem Synchronization

- Synthesized and aligned `training/mnime\_v5\_antibias\_dataset\_clean.jsonl` (20,000 synthetic examples targeting systemic bias, stereotype elimination, and dialectical anti-bigotry).
- Configured MNIME-Core to deconstruct hateful tropes and prejudiced generalizations through sociological rigor and empirical logic rather than shallow, evasive refusal templates.
- Assimilated V5 ethics matrices through the Neural Assimilation Engine into the production weights.
- Consolidated multi-tier model artifacts into `models/` with fallback support and updated `version\_info.txt`.
- Synchronized all documentation, spec sheets, change logs, and research papers across `README.md`, `CHANGE\_LOG.txt`, `MNIME\_Architecture.pdf`, `MNIME\_Change\_Log.pdf`, `MNIME\_Spec\_Manual.pdf`, and `MNIME\_paper.tex`.